

ANNEXE — Parcours C# / .NET 10 / ASP.NET Core

Cette annexe fournit le code de l'application (à recopier tel quel) et les valeurs propres à cette techno.

Si votre poste n'a que le SDK .NET 9, remplacez partout 10.0 par 9.0 et net10.0 par net9.0 : tout le reste est identique.

1. Ce que vous devez savoir avant de commencer

C# est un langage **compilé**. Produire l'application demande le **SDK .NET** (compilateur + NuGet + MSBuild), ~800 Mo, alors que l'exécution ne réclame que le **runtime ASP.NET Core**. Microsoft publie donc deux familles d'images — `sdk` et `aspnet` — précisément pour le build multi-stage. Les utiliser correctement est l'objet de la Partie 1.

2. Arborescence du dossier `api/`

```
api/
├── Dockerfile                ← à écrire
├── .dockerignore            ← à écrire
├── src/TpApi/
│   ├── TpApi.csproj
│   ├── Program.cs
│   └── TitreUtils.cs
└── tests/TpApi.Tests/
    ├── TpApi.Tests.csproj
    └── TitreUtilsTests.cs
```

3. Code fourni

`api/src/TpApi/TpApi.csproj`

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
```

```

    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>TpApi</RootNamespace>
    <!-- C'est CETTE version que la CI devra extraire automatiquement -->
    <Version>0.1.0</Version>
</PropertyGroup>

<ItemGroup>
    <PackageReference Include="Npgsql" Version="9.0.2" />
</ItemGroup>

</Project>

```

api/src/TpApi/TitreUtils.cs

```

using System.Text.RegularExpressions;

namespace TpApi;

/// <summary>Petite règle métier, présente surtout pour donner matière au test
unitaire de la CI.</summary>
public static class TitreUtils
{
    public static string Normaliser(string? titre)
    {
        if (string.IsNullOrEmpty(titre))
            throw new ArgumentException("Le titre ne peut pas être vide",
            nameof(titre));

        return Regex.Replace(titre.Trim(), @"\s+", " ");
    }
}

```

api/src/TpApi/Program.cs

```

using System.Net;
using Npgsql;
using TpApi;

var builder = WebApplication.CreateBuilder(args);

// AUCUNE chaîne de connexion en dur : elle vient de l'environnement,
// via la variable ConnectionStrings__Default (double underscore).
var connectionString = builder.Configuration.GetConnectionString("Default")

```

```

    ?? throw new InvalidOperationException("ConnectionStrings__Default n'est
pas définie");

builder.Services.AddSingleton(_ => NpgsqlDataSource.Create(connectionString));

var app = builder.Build();

// Sonde de santé utilisée par le HEALTHCHECK du Dockerfile
app.MapGet("/health", () => Results.Ok(new { status = "UP" }));

app.MapGet("/api/taches", async (NpgsqlDataSource db) =>
{
    var taches = new List<Tache>();
    await using var cmd = db.CreateCommand("SELECT id, titre, faite FROM tache
ORDER BY id");
    await using var reader = await cmd.ExecuteReaderAsync();
    while (await reader.ReadAsync())
        taches.Add(new Tache(reader.GetInt64(0), reader.GetString(1),
reader.GetBoolean(2)));
    return Results.Ok(taches);
});

app.MapPost("/api/taches", async (NouvelleTache nouvelle, NpgsqlDataSource db)
=>
{
    string titre;
    try { titre = TitreUtils.Normaliser(nouvelle.Titre); }
    catch (ArgumentException e) { return Results.BadRequest(new { erreur =
e.Message }); }

    await using var cmd = db.CreateCommand(
        "INSERT INTO tache (titre, faite) VALUES ($1, $2) RETURNING id");
    cmd.Parameters.AddWithValue(titre);
    cmd.Parameters.AddWithValue(nouvelle.Faite);

    var id = (long)(await cmd.ExecuteScalarAsync())!;
    return Results.Created($"api/taches/{id}", new Tache(id, titre,
nouvelle.Faite));
});

// Renvoie le hostname du conteneur : permet de vérifier le load-balancing.
app.MapGet("/api/qui", () => Results.Text(Dns.GetHostName()));

app.Run();

```

```
public record Tache(long Id, string Titre, bool Faite);  
public record NouvelleTache(string? Titre, bool Faite);
```

api/tests/TpApi.Tests/TpApi.Tests.csproj

```
<Project Sdk="Microsoft.NET.Sdk">  
  
  <PropertyGroup>  
    <TargetFramework>net10.0</TargetFramework>  
    <Nullable>enable</Nullable>  
    <ImplicitUsings>enable</ImplicitUsings>  
    <IsPackable>false</IsPackable>  
  </PropertyGroup>  
  
  <ItemGroup>  
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />  
    <PackageReference Include="xunit" Version="2.9.2" />  
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />  
  </ItemGroup>  
  
  <ItemGroup>  
    <ProjectReference Include="../../src/TpApi/TpApi.csproj" />  
  </ItemGroup>  
  
</Project>
```

api/tests/TpApi.Tests/TitreUtilsTests.cs

```
using TpApi;  
using Xunit;  
  
public class TitreUtilsTests  
{  
    [Fact]  
    public void Normalise_Les_Espaces()  
    {  
        Assert.Equal("faire le TP", TitreUtils.Normaliser(" faire le TP  
"));  
    }  
  
    [Fact]  
    public void Refuse_Un_Titre_Vide()  
    {  
        Assert.Throws<ArgumentException>(() => TitreUtils.Normaliser(" "));  
    }  
}
```

```
}  
}
```

4. Valeurs propres à ce parcours

Élément	Valeur
Image de build	<code>mcr.microsoft.com/dotnet/sdk:10.0-alpine</code>
Image finale	<code>mcr.microsoft.com/dotnet/aspnet:10.0-alpine</code>
Manifeste à copier en premier (cache)	<code>src/TpApi/TpApi.csproj</code> , suivi de <code>dotnet restore</code>
Commande de publication	<code>dotnet publish src/TpApi/TpApi.csproj -c Release -o /app/publish</code>
Artefact à copier depuis l'étape de build	<code>/app/publish</code>
Port	8080 (défaut des images .NET ≥ 8 ; forcez-le avec <code>ASPNETCORE_URLS=http://+:8080</code>)
Dossiers de build locaux à ignorer	<code>bin/</code> , <code>obj/</code>
Sonde de santé	<code>GET http://localhost:8080/health</code>
Commande de lancement	<code>dotnet /app/TpApi.dll</code>
Utilisateur non-root	<code>USER \$APP_UID</code> — variable déjà définie dans les images .NET ≥ 8 (uid 1654). Pas besoin de créer l'utilisateur.
Variable d'environnement de connexion	<code>ConnectionStrings__Default</code>
Valeur de la chaîne	<code>Host=db;Port=5432;Database=<base>;Username=<user>;Password=<mdp></code>
Commande de test (job CI test)	<code>dotnet test</code>
Action de setup en CI	<code>actions/setup-dotnet@v4</code> avec <code>dotnet-version: '10.0.x'</code> , plus <code>cache: true</code> et <code>cache-dependency-path</code>
Extraction de la version	<code>dotnet msbuild src/TpApi/TpApi.csproj -getProperty:Version</code>
Objectif de taille	< 200 Mo

5. Pièges spécifiques à .NET

- **Le double underscore** `ConnectionStrings__Default` **n'est pas une faute de frappe**. C'est la convention .NET pour représenter une configuration hiérarchique (`ConnectionStrings:Default`) dans une variable d'environnement, où `:` est interdit sous Linux. Avec un seul underscore, la configuration n'est pas trouvée et l'application refuse de démarrer.
- `dotnet restore` **dans l'étape de cache**, mais attention : le projet de test référence le projet applicatif. Ne copiez **que** `src/TpApi/TpApi.csproj` dans l'image — les tests tournent dans la CI, pas dans le build de l'image.
- `USER $APP_UID` **doit être placé après les** `COPY`, sinon les fichiers appartiennent à root et l'application peut ne pas pouvoir lire son propre répertoire. Utilisez `COPY --from=build --chown=$APP_UID:$APP_UID ...`.
- `dotnet --list-sdks` **doit être vide** dans l'image finale : c'est votre preuve que le SDK n'a pas fui dans l'image de production. Si la commande liste un SDK, votre `FROM` final est le mauvais.
- **Alpine et l'ICU** : les images `-alpine` embarquent une version réduite d'ICU. Si vous voyez une erreur *globalization*, ajoutez `ENV DOTNET_SYSTEM_GLOBALIZATION_INVARIANT=1` ou basculez sur l'image finale non-alpine (plus lourde — à mentionner dans le rapport).
- `curl` **n'existe pas** dans les images `aspnet:*-alpine` : utilisez `wget` (busybox) pour le `HEALTHCHECK`.

6. Vérification rapide en local (hors Docker)

```
cd api
dotnet test # doit être vert avant même d'écrire le Dockerfile
```